

Aula 2 — De C à máquina

Três programas, uma listagem e um som

Microcontroladores — DENE/UFMT

OBJETIVOS

- Ler a listagem gerada pelo compilador, associando cada instrução de máquina à linha de C que a originou.
- Calcular o tempo de execução de um trecho por contagem de ciclos, distinguindo instruções de um ciclo das de dois e do desvio condicional.
- Explicar por que `__delay_ms` exige constante e construir uma rotina de atraso cuja duração é decidida em tempo de execução.
- Calibrar essa rotina com o osciloscópio e converter uma nota musical em um número inteiro de iterações, sem ponto flutuante.
- Distinguir, pelo som, um erro de constante de um erro de sobrecarga.

1 Três programas

Esta aula tem três programas e um objetivo. Os programas são: apagar um LED, piscar um LED, tocar uma melodia. O objetivo é que o terceiro deixe de ser mágica.

Programa	A pergunta que ele obriga a responder
1 Apaga um LED e para	O que exatamente o compilador produziu, e em que instante o LED apaga
2 Pisca um LED	Quanto tempo dura um trecho de código, e por que o atraso da biblioteca não serve
3 Toca uma melodia	Como converter uma frequência desejada num número inteiro

Nenhum dos três é difícil. O que é novo é olhar para o que o compilador fez com eles.

1.1 O que muda quando o destino é um microcontrolador

Vocês já escreveram C para computador. O mesmo compilador, apontado para outro destino, produz algo com três diferenças visíveis já no primeiro programa.

No computador	Aqui	Consequência no código
Existe sistema operacional	Não existe	Não há para onde retornar. <code>main</code> não termina: termina em um laço infinito, ou o processador continua buscando instruções no que vier depois
Existe <code>printf</code>	Não existe	A saída é um pino. Escrever num pino é escrever numa posição de memória
O programa é carregado na RAM	O programa mora na Flash	Variáveis inicializadas precisam ser copiadas da Flash para a RAM antes de <code>main</code> — por um código que ninguém escreveu

CONCEITO

O compilador é um tradutor, e o destino dita o vocabulário. O mesmo `a + b` em C vira, num processador de mesa, uma operação entre registradores de um banco de dezesseis. Aqui vira uma sequência sobre um único acumulador chamado `WREG`, operando direto sobre endereços de memória.

Não é que uma versão seja melhor. É que a frase em C não contém a informação que distingue as duas — e tudo que esta aula vai medir mora na diferença.

2 Programa 1: apagar um LED

2.1 A chave que já acendeu os LEDs

Antes de gravar coisa alguma: ligue a chave que conecta o painel de LEDs ao PORTD. Os LEDs acendem.

Isso aconteceu com o processador em *reset*, com todos os pinos em alta impedância, e sem que uma única linha de C tenha executado. Vale parar aqui.

CONCEITO

Nada no programa diz que RD0 é um LED. Essa informação não está no `xc.h`, não está na folha de dados do PIC18F4550, não está no compilador. Ela está numa chave de duas posições no painel — e o estado dessa chave é o que decide se aquele pino comanda um LED ou não comanda nada.

É a primeira camada do semestre que nenhuma ferramenta conhece. Voltaremos a ela na §2.7 com nome e endereço.

2.2 O programa

```
#include <xc.h>

void main(void)
{
    LATDbits.LATD0 = 0; /* valor no latch */
    TRISDbits.TRISD0 = 0; /* so' entao saida */
    while (1) { }
}
```

Três atribuições, e a terceira nem é atribuição. Traduzido, é a listagem inteira — três instruções, oito bytes de Flash:

```
000800 908C      bcf  140,0,c      ; LATD bit 0 <- 0
000802 9095      bcf  149,0,c      ; TRISD bit 0 <- 0
000804 EF02 F004  goto $           ; while (1)
```

Instrução	O que faz, e quando o LED apaga
<code>bcf 140,0,c</code>	<i>Bit clear file.</i> Zera o bit 0 do endereço 140, que é <code>LATD</code> . Escreve 0 no <i>latch</i> de RD0 — e nada acontece eletricamente . O pino ainda é entrada, em alta impedância, e o LED continua aceso. O valor fica guardado, esperando
<code>bcf 149,0,c</code>	Zera o bit 0 de <code>TRISD</code> ; zero significa saída. RD0 vira saída e assume o que já estava no <i>latch</i> . O LED apaga aqui
<code>goto \$</code>	O <code>\$</code> é notação do montador para “o endereço desta própria instrução”. Um desvio para si mesma, dois ciclos, indefinidamente. É o <code>while (1)</code> do C — e é o que impede o processador de sair buscando instruções no vazio

CONCEITO

A regra LAT antes de TRIS, reduzida a dois opcodes. Ela não é convenção de estilo nem recomendação do fabricante: é a ordem de duas instruções, e o observável é o instante em que o LED apaga.

Invertendo as duas linhas, o pino vira saída assumindo o que estiver no *latch* naquele momento — que o seu programa ainda não escreveu. Com um LED o resultado é um pulso curto. Com o *cooler*, é um tranco no motor.

EXPERIMENTO

Em aula. Rode as duas versões no simulador, com `--pino RD0`, e compare o instante da transição. Uma tem a borda na segunda instrução; a outra, na primeira — e um pulso indesejado antes dela.

2.3 Uma instrução decodificada à mão

Vale fazer isto uma vez na vida, porque desfaz o mistério de forma definitiva. A primeira instrução da listagem é `0x908C`. O manual do dispositivo dá o formato de `BCF` — desligar um bit — como `1001 bbba ffffffff`. Escrevendo `0x908C` em binário:

$$\underbrace{1001}_{\text{código}} \underbrace{000}_b \underbrace{0}_a \underbrace{10001100}_f$$

Campo	Valor	Significado
<i>b</i>	<code>000</code> = 0	O bit a desligar é o bit 0
<i>a</i>	<code>0</code>	Endereço interpretado no banco de acesso; vide §4.3
<i>f</i>	<code>10001100</code> <code>0x8C</code>	= Endereço 140 — que é LATD

Ou seja: `LATDbits.LATD0 = 0`; compilou para **uma única instrução de dois bytes**, que executa em 250 ns e zera o bit 0 do endereço `0x8C`.

CONCEITO

Isto é a afirmação da Aula 1, §3 — “os pinos são posições de memória” — já não como analogia, mas como campo de bits dentro de um opcode. O endereço 140 não é tratado de maneira especial em lugar nenhum: é o mesmo campo *f* que apontaria para uma variável comum.

O que faz `0x8C` acionar um LED é o silício ligado atrás dele e a chave do painel, não a instrução.

2.4 Cinco instruções, por enquanto

O conjunto completo do PIC18 tem 75 instruções. Não vamos vê-las. Vamos começar com cinco e acrescentar uma linha cada vez que um programa exigir.

Instrução	Efeito	Ciclos
<code>bcf f, b</code>	Desliga o bit <i>b</i> de <i>f</i>	1
<code>bsf f, b</code>	Liga o bit <i>b</i> de <i>f</i>	1
<code>movlw k</code>	$W \leftarrow k$ (carrega constante no acumulador)	1
<code>movwf f</code>	$f \leftarrow W$ (escreve o acumulador na memória)	1
<code>goto k</code>	Desvio incondicional	2

CONCEITO

O que caracteriza este núcleo não é a quantidade de instruções, e sim três decisões:

Um acumulador só `WREG`. Não há banco de registradores.

Operação direta sobre a memória `bcf 140, 0` altera a memória sem carregar nada antes. É o oposto de uma arquitetura *load-store*, em que a memória só é tocada por instruções de carga e armazenamento.

Largura fixa toda instrução ocupa uma palavra de dois bytes, exceto `goto`, `call`, `lfsr` e `movff`, que ocupam duas palavras.

Ligar um bit de porta custa uma instrução aqui e três num Cortex-M — que ainda assim termina antes, por operar a uma frequência muito maior. Contagem de instruções não é medida de desempenho.

2.5 Por que o programa começa em `0x0800`

Repare no primeiro endereço da listagem: não é zero. Os primeiros 2 KB da Flash estão ocupados pelo *bootloader*, o programa que recebe o seu `.hex` pela USB e o grava. O seu código é deslocado para começar depois dele.

ATENÇÃO

Duas consequências que valem o semestre inteiro:

1. **O *bootloader* é dono dos bits de configuração.** Um `#pragma config` no seu código é aceito pelo compilador e ignorado na prática — ele foi gravado antes, por outro programa. Quem manda no relógio não é você.
2. **O relógio é de 16 MHz**, não de 48 MHz. Os 48 MHz aparecem na documentação porque são exigência do módulo USB, que é o que o *bootloader* usa. O núcleo executa a $\frac{F_{osc}}{4}$, ou seja, um ciclo de máquina a cada 250 ns. Todo número desta aula sai daí.

$$T_{cy} = \frac{4}{F_{osc}} = \frac{4}{16 \text{ MHz}} = 250 \text{ ns}$$

NOTA

A listagem completa deste programa tem mais de 600 linhas, das quais apenas três são instruções. O resto são definições `equ` — um nome para cada registrador do dispositivo, inclusive os do módulo USB que o programa nunca toca — e diretivas do montador e do ligador. Não custam Flash nenhuma.

Programas com variáveis globais inicializadas ganham, além disso, um bloco de partida antes de `main`, que copia os valores iniciais da Flash para a RAM com a instrução `TBLRD`. A separação Harvard da Aula 1, §4, que naquela aula era uma consequência a acreditar, é esse laço de cópia.

2.6 Do nome ao pino: quatro camadas

O programa diz `LATDbits.LATD0`; a listagem diz `140`; a bancada tem um LED. Entre um extremo e outro há quatro camadas, e apenas a primeira é software.

Camada	Onde vive, e o que pode mudá-la
1 Nome → endereço	No cabeçalho do dispositivo, incluído por <code>xc.h</code> . É uma declaração da forma <code>extern volatile unsigned char LATD __at(0xF8C)</code> ; mais a união de campos de bits que dá <code>LATDbits.LATD0</code> . Pura tabela de nomes
2 Endereço → bloco interno	Fixo em silício. Documentado no mapa de registradores da folha de dados. Nenhum ajuste de software altera
3 Bloco interno → pino físico	Fixo em silício. RD0 é o pino 19 do encapsulamento de 40 pinos. Está no diagrama de pinagem
4 Pino físico → o que há na placa	Não existe em arquivo nenhum. Está no esquemático da XM118, na serigrafia e na posição das chaves — e a serigrafia tem precedência sobre o manual

ATENÇÃO

A camada 4 é a que interessa na bancada, e é a única que nenhuma ferramenta conhece. Foi ela que acendeu os LEDs no começo da aula, antes de qualquer programa. E é ela que decide, mais adiante, se RC2 vai ao *buzzer* ou ao *cooler*.

O compilador não avisa que `LATD = 0x01` desliga alguma coisa importante porque a informação “o bit 1 deste endereço comanda 12 V” não existe em lugar algum que ele possa ler. Ela mora no esquema elétrico e na cabeça de quem escreveu.

3 O simulador

Vamos rodar os três programas num simulador escrito em Python, que lê o mesmo `.hex` que o gravador recebe. Ele não é uma aproximação do PIC: ele conta ciclos.

```
python3 pic18.py apaga.hex --pino RD0 --passos 20 --trace
```

Opção	O que faz
<code>--trace</code>	Imprime, a cada instrução, o endereço, o opcode, o mnemônico e o acumulador
<code>--dump</code>	Despeja a região de memória ao final — útil quando não há saída visível
<code>--passos N</code>	Executa no máximo <i>N</i> instruções; sem isso, <code>goto \$</code> roda para sempre
<code>--pino P</code>	Registra as transições do pino <i>P</i> com o instante de cada uma, em ciclos

Uma saída típica de `--trace`:

```
0800  908C  bcf    LATD,0,c      W=00  ciclo 1
0802  9095  bcf    TRISD,0,c     W=00  ciclo 2      <-- RD0: 1 -> 0
0804  EF02  goto   0x804      W=00  ciclo 3
```

CONCEITO

Por que um simulador, se existe a placa. Porque ele responde a perguntas que a placa não responde. Quantos ciclos exatamente durou aquele laço? Qual instrução estava executando quando o pino mudou? Na bancada isso exige instrumento e interpretação; aqui é uma contagem.

Placa e simulador são fontes independentes. Quando discordam, aprendeu-se algo — e é por isso que o Roteiro 2 mede com o osciloscópio o que hoje calculamos aqui.

4 Programa 2: piscar um LED

Piscar é apagar, esperar, acender, esperar. A parte nova é “esperar”.

```
#include <xc.h>
#define _XTAL_FREQ 16000000UL

void main(void)
{
    LATDbits.LATD0 = 0;
    TRISDbits.TRISD0 = 0;
    while (1) {
        LATDbits.LATD0 ^= 1;
        __delay_ms(500);
    }
}
```

ATENÇÃO

`_XTAL_FREQ` é um número que **você** declara e que o compilador não tem como conferir. Se estiver errado, `__delay_ms(500)` compila sem um aviso sequer e espera outra coisa. Declare 48000000 aqui e o LED pisca três vezes mais devagar do que você pediu.

Guarde este mecanismo. Ele volta, com consequência bem pior, quando a comunicação serial entrar em cena.

4.1 A constante que não varia

Abra a listagem do laço. O que `__delay_ms(500)` virou é um par de laços aninhados com números literais dentro:

```
    movlw 10                ; contador externo
    movwf ??_main+2,c
u1: movlw 200               ; contador do meio
    movwf ??_main+1,c
u2: movlw 250               ; contador interno
    movwf ??_main,c
u3: decfsz ??_main,f,c
    goto u3
    decfsz ??_main+1,f,c
    goto u2
    decfsz ??_main+2,f,c
    goto u1
```

CONCEITO

Onde está o 500? Em lugar nenhum. Está dissolvido em três constantes literais, gravadas na Flash pelo compilador. Não há variável, não há parâmetro, não há como um outro valor entrar ali em tempo de execução.

A razão é que `__delay_ms` **não é uma função**. É uma macro:

```
#define __delay_ms(x) _delay((unsigned long)((x)*(_XTAL_FREQ/4000.0)))
```

Uma macro é substituição textual, feita pelo pré-processador antes de o compilador existir. O argumento `x` é copiado para dentro de uma expressão que precisa ser avaliada na hora da compilação. Passar uma variável é impossível por construção: no instante em que a substituição acontece, variáveis ainda não existem.

NOTA

Não é um defeito da biblioteca — é a única forma de gerar um atraso exato sem gastar RAM nem ciclos calculando. O preço é a rigidez. Para o Programa 3 precisamos do oposto: um atraso cuja duração é decidida enquanto o programa roda.

Existe outra família de macros, aquelas com corpo de várias instruções, cuja escrita correta tem armadilhas próprias. Isso é assunto do Exercício 2.6.

4.2 Um ciclo por instrução, exceto no desvio

A Aula 1, §8.3, afirmou que programa e dados em barramentos separados permitem buscar a próxima instrução enquanto a atual executa, e que por isso o processador completa uma instrução por ciclo. Agora dá para usar isso.

CONCEITO

Por que o desvio custa o dobro. Enquanto a instrução no endereço n executa, a do endereço $n + 2$ já está sendo buscada. Se a instrução em n for um desvio, a próxima a executar não é a de $n + 2$: a busca adiantada foi trabalho perdido, e o processador gasta um ciclo adicional buscando o destino verdadeiro.

Não é penalidade nem defeito. É o preço de um mecanismo que, no resto do tempo, entrega uma instrução por ciclo de graça.

Instrução	Ciclos	Observação
Aritmética, lógica, movimentação, bit	1	A maioria esmagadora do código
<code>goto</code> , <code>call</code> , <code>return</code> , <code>bra</code>	2	Sempre; o desvio é sempre tomado
<code>decfsz</code> / <code>btfsz</code> / <code>btfss</code> que não pula	1	Segue em frente; nada foi descartado
<code>decfsz</code> / <code>btfsz</code> / <code>btfss</code> que pula uma instrução de uma palavra	2	Descarta a busca adiantada
<code>decfsz</code> / <code>btfsz</code> / <code>btfss</code> que pula uma instrução de duas palavras	3	Precisa descartar as duas palavras

ATENÇÃO

A última linha é a mais esquecida, e `goto` é justamente uma instrução de duas palavras. Um desvio condicional que pula um `goto` custa três ciclos. Como o XC8 sem otimização gera exatamente esse padrão o tempo todo, o erro se acumula rápido em qualquer contagem manual.

Duas instruções novas para a lista:

Instrução	Efeito	Ciclos
<code>decfsz f, f</code>	Decrementa f ; pula a próxima instrução se o resultado for 0	1, 2 ou 3
<code>movf f, w</code>	$W \leftarrow f$; afeta o indicador de zero	1

EXPERIMENTO

Conte os ciclos do laço interno u3 acima: `decfsz` mais `goto`. Multiplique por 250 ns e por 250 voltas. Compare com o que o simulador reporta. A diferença entre a sua conta e a contagem dele é onde está o que você esqueceu.

4.3 O sufixo `,c`

Quase toda instrução da listagem termina em `,c`. É o bit a do opcode, aquele que a §2.4 decodificou como zero.

A Aula 1, §8.4, explicou que os 2 048 bytes de RAM são divididos em bancos de 256 porque o campo de endereço da instrução tem apenas 8 bits, e que o PIC18 mantém uma janela de 256 bytes reunindo os 96 primeiros bytes de RAM aos registradores de periférico do topo do mapa. O `,c` diz que o endereço foi resolvido nessa janela:

uma instrução, um ciclo. Sem ele, o compilador precisa emitir antes um `movlb` — mais uma palavra de Flash e mais um ciclo.

ATENÇÃO

A janela comporta 96 bytes de RAM. Um projeto que ultrapasse isso em variáveis frequentemente acessadas começa a ter parte delas fora — e o código que as usa fica mais lento **sem que nenhuma linha desse código tenha mudado**.

É o primeiro caso do semestre em que acrescentar uma variável em um módulo altera a temporização de outro. Quando uma rotina calibrada começar a errar depois de o projeto crescer, esta é a hipótese. O Exercício 2.5 trabalha o caso.

5 A duração como parâmetro

```
void atraso_unidades(uint16_t unidades)
{
    while (unidades > 0u) {
        unidades--;
    }
}
```

Nada nesse código expressa duração. Não há microssegundo nenhum escrito ali. A duração é propriedade do que ele virou, e a listagem informa quanto custa uma volta.

5.1 O modelo

CONCEITO

Chamando a rotina com N unidades, o tempo total é

$$t(N) = (a + b \cdot N) \cdot T_{cy}$$

com $T_{cy} = 250$ ns. É uma reta: b é o custo de uma iteração, em ciclos, e a é a sobrecarga que se paga uma vez — `call`, passagem do parâmetro, teste final, `return`.

Duas grandezas, duas incógnitas. Por isso **duas** medições determinam ambas experimentalmente, sem abrir a listagem — e é assim que a contagem pode ser confrontada com a bancada.

EXPERIMENTO

Procedimento — e é ele que vocês repetem no Roteiro 2.

1. Compilar e abrir a listagem. Localizar o comentário com o número da linha do `while` e ler o bloco até o `goto` que volta ao início.
2. Anotar cada instrução com seu custo, pela tabela da §4.2. Atenção aos desvios condicionais que pulam `goto`.
3. Somar. O resultado é b .
4. Somar separadamente o que executa uma única vez. O resultado é a .
5. Conferir os dois no simulador, que conta sem opinião.

NOTA

A chamada custa. Dos ciclos de a , quatro são apenas `call` mais `return`, sem que nenhum trabalho útil aconteça.

Onde vai o endereço de retorno: numa arquitetura convencional, o `call` empilha na mesma RAM onde vivem as variáveis. Aqui não — a pilha de retorno é um bloco de silício dedicado, com 31 níveis, fora do

espaço de dados (Aula 1, §8.2). A consequência agradável é que chamadas aninhadas não consomem os 2 KB de RAM; a desagradável é que a profundidade é fixa em 31, o que só é problema com recursão.

5.2 Onde a conta não fecha exatamente

O contador é `uint16_t` e o núcleo é de 8 bits. Decrementar exige tratar o byte alto — mas o byte alto só muda quando o byte baixo passa por zero, uma vez a cada 256 iterações. O compilador gera esse tratamento como um desvio condicional, e o custo da iteração depende de qual caminho foi tomado.

ATENÇÃO

A consequência é que *b não é um número inteiro*. É uma média. Contando a listagem você chega a 9 ciclos; medindo, chega-se a um pouco mais, e a diferença é o caminho que executa uma vez a cada 256 voltas.

Isso não é erro de contagem. É o que a contagem manual honestamente não pode entregar, e é a primeira razão pela qual vamos medir em vez de só contar.

6 Programa 3: da frequência à nota

Alternando o pino a cada atraso, o programa gera uma onda quadrada:

```
while (1) {  
    BUZZER ^= 1;  
    atraso_unidades(N);  
}
```

Cada chamada ocupa **meio** período: o pino sobe, espera, desce, espera. Este é o ponto onde a maioria dos erros nasce, então vale escrever devagar.

6.1 Meio período, não período

CONCEITO

Uma onda quadrada de frequência f tem período $T = 1/f$. O pino passa metade desse tempo em nível alto e metade em nível baixo. Cada chamada de `atraso_unidades` produz **uma** dessas metades.

Chamando de H o meio período:

$$H = \frac{T}{2} = \frac{1}{2f}$$

Para f em hertz e H em microssegundos:

$$H = 500 \frac{000}{f}$$

Uma nota lá de 440 Hz tem $H = 500\,000/440 = 1136,4\,\mu\text{s}$. É esse o número que a rotina de atraso precisa produzir — não os 2 272,7 μs do período inteiro.

ATENÇÃO

Trocar H por T é o erro mais comum aqui, e ele é audível: a nota sai uma oitava abaixo, porque metade da frequência é exatamente um intervalo de oitava. Se a melodia inteira soar grave demais, mas certa entre si, suspeite disto antes de qualquer outra coisa.

6.2 Calibração com o osciloscópio

O modelo tem duas incógnitas. Duas medições as determinam. Ligue a ponta de prova no pino, grave duas versões com valores diferentes de N e leia a frequência.

Duas medidas feitas nesta bancada:

N	Frequência medida	Meio período $H = 500\,000/f$
400	551,0 Hz	907,44 μs
800	276,2 Hz	1810,28 μs

Como $H = a + bN$, a inclinação sai da diferença e o intercepto sai de qualquer um dos pontos:

$$b = \frac{H_2 - H_1}{N_2 - N_1} = \frac{1810,28 - 907,44}{800 - 400} = 2,2571 \text{ } \mu\text{s}$$

$$a = H_1 - bN_1 = 907,44 - 2,2571 \cdot 400 = 4,60 \text{ } \mu\text{s}$$

CONCEITO

O que os dois números dizem.

$b = 2,2571 \text{ } \mu\text{s}$ são 9,03 ciclos. A contagem na listagem deu 9. A fração de 0,03 é exatamente o tratamento do byte alto, que executa uma vez a cada 256 voltas: $8/256 \approx 0,03$, coerente com um caminho adicional de cerca de 8 ciclos. **A contagem estava certa, e a medida disse o que ela não podia dizer.**

$a = 4,60 \text{ } \mu\text{s}$ são 18,4 ciclos: `call`, `return`, a passagem do parâmetro, o teste que encerra o laço, a inversão do pino e o `goto` do laço externo. Ordem de grandeza compatível com o que se conta.

NOTA

Por que duas medidas, e não uma. Com um ponto só, para achar b é preciso supor $a = 0$. Com $N = 400$ isso erraria em 0,5%; com $N = 20$, erraria em 10%. A sobrecarga é irrelevante para atrasos longos e decisiva para os curtos — que são justamente as notas agudas.

E uma terceira? Não melhora o ajuste: dois parâmetros já estão determinados por duas equações. Uma terceira medida **testa** o modelo. Se o ponto previsto cair fora da tolerância do instrumento, a hipótese de linearidade está errada, e a resposta interessante deixa de ser refinar a e b e passa a ser descobrir o que é não linear.

6.3 Da nota ao N , sem ponto flutuante

Agora o caminho inverso: dada a frequência desejada, qual N ?

$$N = \frac{H - a}{b} = \frac{500\,000/f - a}{b}$$

Para o lá de 440 Hz, com $a = 4,60$ e $b = 2,2571 \text{ } \mu\text{s}$:

$$N = \frac{1136,36 - 4,60}{2,2571} = 501,3 \rightarrow 501$$

Só que a licença gratuita do XC8 não tem ponto flutuante utilizável, e nós não queremos pagar por ele. A saída é a mesma das temperaturas em décimos de grau da Aula 1, §7.4: **escolher uma unidade menor e trabalhar com inteiros.**

CONCEITO

Ponto fixo é isto e nada mais: guardar 2,2571 μs como o inteiro 23 e lembrar que a unidade é o décimo de microssegundo. Ou como o inteiro 226, lembrando que a unidade é o centésimo.

Nenhuma operação muda. O que muda é quanto da grandeza real sobrevive ao arredondamento — e essa escolha, aqui, é audível.

6.4 Décimos ou centésimos

Compare as duas representações do mesmo $b = 2,2571 \mu\text{s}$:

Unidade	b guardado	Valor implicado	Erro relativo
décimo de μs	23	$2,3 \mu\text{s}$	+1,9%
centésimo de μs	226	$2,26 \mu\text{s}$	+0,13%

Refazendo a conta do lá em cada unidade — com H e a na mesma unidade de b , para que a divisão seja de inteiros:

Unidade	Conta	N	f real	Erro
décimos	$(11\ 364 - 46)/23$	492	448,39 Hz	+8,4 Hz
centésimos	$(113\ 636 - 460)/226$	501	440,37 Hz	+0,4 Hz

ATENÇÃO

Oito hertz num lá é meio tom mal contado. Um instrumento afinado assim soa errado ao lado de qualquer outro.

E note onde o erro nasceu: não na medida, que tem quatro algarismos significativos, e não na contagem, que estava certa. Nasceu na **representação** — em jogar fora, no arredondamento, mais precisão do que o resultado tolera. Medir com quatro algarismos e guardar dois é trabalho desperdiçado.

6.5 Quanto é “errado” em música

Hertz não é a unidade certa para julgar isto, porque o ouvido responde a razões, não a diferenças. Oito hertz num lá de 440 é bastante; oito hertz num dó de 523 Hz é menos; oito hertz numa nota de 4 kHz é nada.

CONCEITO

A unidade musical do erro é o **cent**: a centésima parte de um semitom, e portanto a $1/1200$ de uma oitava.

$$\text{cents} = 1200 \log_2 \left(\frac{f_{\text{real}}}{f_{\text{alvo}}} \right)$$

Para erros pequenos, o logaritmo é dispensável:

$$\text{cents} \approx 1731 \cdot \frac{\Delta f}{f}$$

porque $1200 / \ln 2 = 1731$. Um erro de 1% em frequência é 17 cents, sempre, em qualquer nota.

Referências: 100 cents é um semitom. Ouvintes treinados percebem de 5 a 10 cents em comparação direta. Abaixo de 5 cents, ninguém ouve.

Aplicando às duas representações:

Unidade	Erro em cents	Consequência
décimos	+33	Um terço de semitom. Claramente audível ao lado de uma referência
centésimos	+1,5	Abaixo do limiar. Ninguém percebe

6.6 A tabela

Calculados uma vez, em centésimos de microssegundo, os oito valores da escala:

Nota	f alvo (Hz)	N	f real (Hz)	Erro (cents)
dó ₄	261,63	844	261,84	+1,4

ré ₄	293,66	751	294,17	+3,0
mi ₄	329,63	669	330,12	+2,6
fá ₄	349,23	631	349,94	+3,5
sol ₄	392,00	562	392,74	+3,3
lá ₄	440,00	501	440,37	+1,5
si ₄	493,88	446	494,43	+1,9
dó ₅	523,25	421	523,65	+1,3

NOTA

As frequências alvo vêm do temperamento igual, em que cada semitom é uma razão fixa: $f_n = 440 \cdot 2^{n/12}$, com n contado em semitons a partir do lá. Doze semitons dobram a frequência, que é a definição de oitava.

CONCEITO

A divisão desapareceu do microcontrolador. A tabela foi calculada uma vez, fora da placa, e entra no programa como constantes. Não há divisão em tempo de execução, não há ponto flutuante, e o custo em RAM é zero:

```
const uint16_t nota_n[8] = { 844, 751, 669, 631, 562, 501, 446, 421 };
```

O qualificador `const` faz a tabela viver na Flash, não na RAM — Harvard outra vez, agora a seu favor. Dezesesseis bytes de Flash, que sobra, em vez de dezesesseis bytes de RAM, que não sobra.

6.7 O programa

```
#include <xc.h>
#include <stdint.h>

#define BUZZER LATCbits.LATC2

const uint16_t nota_n[8]      = { 844, 751, 669, 631, 562, 501, 446, 421 };
const uint16_t nota_meios[8] = { 131, 147, 165, 175, 196, 220, 247, 262 };

void atraso_unidades(uint16_t unidades)
{
    while (unidades > 0u) unidades--;
}

void toca(uint16_t n, uint16_t meios_periodos)
{
    while (meios_periodos > 0u) {
        BUZZER ^= 1;
        atraso_unidades(n);
        meios_periodos--;
    }
}

void main(void)
{
    uint8_t i;
    LATCbits.LATC2 = 0;
    TRISCbits.TRISC2 = 0;
    for (i = 0; i < 8; i++)
```

```
toca(nota_n[i], nota_meios[i]);
while (1) { }
}
```

NOTA

A segunda tabela existe porque a duração de uma nota é contada em meios períodos, e meios períodos são mais curtos nas notas agudas. Para que todas durem 250 ms, o número de meios períodos precisa ser $250\,000/H$ — mais para as agudas, menos para as graves. O Exercício 2.3 explora o que acontece se essa tabela não existir.

6.8 O que se ouve, e o que isso diagnostica**EXPERIMENTO**

Em aula. Rode com a tabela em centésimos e, em seguida, com a tabela em décimos. Compare cada uma com um lá de referência.

Preencham antes de ouvir:

Versão	O que você espera ouvir
centésimos	
décimos	

Um detalhe da versão em décimos merece atenção: o erro de +33 cents é **praticamente o mesmo em todas as oito notas**. A melodia sai afinada consigo mesma, apenas transposta um terço de semitom para cima. Só uma referência externa denuncia.

CONCEITO

A assinatura do defeito. Isto não é curiosidade — é o método de diagnóstico desta aula, e ele funciona porque os dois erros possíveis têm formas diferentes.

Se o erro for	A assinatura é	Porque
na constante b	uniforme em cents, todas as notas na mesma direção	N é proporcional a $1/b$, e o tempo real é proporcional a N : o fator de erro é o mesmo para todo mundo
numa sobrecarga fixa por meio período	cresce com a altura da nota	um número fixo de microssegundos a mais é uma fração pequena de um meio período grave e uma fração grande de um agudo

Ouvir **qual** das duas está acontecendo custa dez segundos e substitui uma tarde de tentativa e erro. É a diferença entre medir e adivinhar.

ATENÇÃO

A segunda linha da tabela não é hipotética neste programa. A função `toca` acrescenta, a cada meio período, a inversão do pino, o decremento de `meios_periodos` e o teste do laço — que não estavam presentes quando a e b foram calibrados com a onda quadrada simples.

Uma constante de calibração vale para o laço em que foi medida. Mudou o que há dentro do laço, recalibre.

7 O limite do método

Fonte de erro	Ordem	Comentário
Byte alto do contador	fração de ciclo	Diluída em N grande; visível em N pequeno
Quantização da constante	até 2%	Decidida por você ao escolher a unidade
Sobrecarga do laço chamador	depende	Cresce com a altura da nota
Tolerância do cristal	0,001% a 0,01%	Desprezível aqui; decisiva na comunicação serial
Interrupções	sem limite	Um tratador executando durante o atraso o estende pelo tempo que levar. Aula 9

CONCEITO

Contagem de ciclos entrega precisão da ordem de 1% e nenhuma garantia. Qualquer coisa que interrompa o laço estraga a conta, e o processador fica integralmente ocupado durante a espera — não pode ler um botão, não pode responder a nada.

A saída não é contar melhor: é deixar de contar. Um temporizador por *hardware* conta em paralelo, sem consumir instruções e sem ser afetado pelo que o programa faz. É o assunto do Encontro 7, e a razão de ele existir.

DIVERGÊNCIA

A rotina construída hoje não é descartada quando o temporizador chegar. Ela permanece útil onde o temporizador é caro demais: esperas de poucos microssegundos em protocolos de sensor, em que armar e desarmar o temporizador custa mais do que a própria espera. Saber contar ciclos continua valendo; o que muda é a escala em que se aplica.

8 Transposição

O instrumento existe em qualquer plataforma, com outro nome.

Ambiente	Como obter a tradução
XC8 / PIC18	O arquivo <code>.lst</code> do diretório de construção, ou <code>xc8-cc -S</code>
GCC, qualquer alvo	<code>gcc -S</code> , ou <code>objdump -d</code> sobre o binário já ligado
ARM Cortex-M	<code>arm-none-eabi-objdump -d</code> ; no CubeIDE, a janela de desmontagem

CONTEXTO

O que **não** se transpõe é a contagem. No PIC18, uma instrução simples custa um ciclo, sempre, e a tabela da §4.2 cabe em cinco linhas. Num Cortex-M típico, o mesmo trecho pode custar diferente conforme o alinhamento na Flash, o número de estados de espera da memória naquela frequência, e a presença de um acelerador de leitura.

Contar ciclos deixa de ser confiável, e é por isso que aquelas plataformas oferecem um contador de ciclos em *hardware* para medir o que já não se consegue prever. Previsibilidade é uma propriedade que se perde ao subir de porte, e ela reaparece no seminário comparativo.

9 O pino que não é digital

CONTEXTO

O Programa 1 tem três instruções porque escolhemos o PORTD.

Escreva o mesmo programa no PORTB e ele deixa de ter três instruções: RB0 a RB4 sobem como **entrada analógica**, e antes de mexer no TRIS é preciso desligar isso num registrador de configuração. A diferença não está no seu código, não está no LED e não está na chave do painel. Está num bit que decide se aquele pino é um bit ou é uma tensão.

Um pino digital só sabe responder “acima ou abaixo do limiar”. Um pino analógico responde “quanto”. O termostato do semestre precisa da segunda resposta, porque um sensor de temperatura não entrega bits.

O próximo encontro é sobre essa decisão.

10 Antes de descer para o laboratório

O Roteiro 2 mede, com o osciloscópio, o que esta aula calculou. Preencham a tabela agora, com a teoria fresca, e levem rubricada.

NOTA

Vale o mesmo da Aula 1: previsão errada não é problema; previsão feita depois do experimento é.

Pergunta	Previsão e justificativa	Ru- brica
P1. Quantos ciclos custa uma iteração do laço de <code>atraso_unidades</code> ? Conte pela listagem antes de medir		
P2. Com $N = 400$, que frequência você espera no pino?		
P3. Dobrando N para 800, a frequência cai exatamente pela metade? Justifique em termos de a e b		
P4. Se o valor medido de b vier maior que o contado, onde estão os ciclos que faltam na sua conta?		
P5. O que aconteceria com a medida se houvesse uma interrupção periódica ativa?		

BANCADA

Leve para a bancada: esta folha preenchida, a listagem impressa ou aberta, e a contagem de ciclos já feita. Medir sem previsão transforma o roteiro em digitação.

11 Exercícios

TAREFA

Exercício 2.1. Um colega roda a melodia e ela sai desafinada. Ele tem um afinador e mede o erro de cada nota, em cents:

Nota	Caso A	Caso B
dó ₄	−31	−6
sol ₄	−33	−14
dó ₅	−32	−19

- (a) Em cada caso, o defeito está na constante b ou numa sobrecarga fixa por meio período? Justifique pela forma dos números, não pelo valor.
- (b) No caso que aponta para a constante, estime o valor correto de b sabendo que o programa usou 23 décimos de microssegundo.
- (c) No outro caso, estime a sobrecarga em microssegundos usando a nota dó₅ ($H = 955,6 \mu\text{s}$).

TAREFA

Exercício 2.2. Pela tabela da §6.6, $N = 844$ para dó₄ e $N = 421$ para dó₅, uma oitava acima.

- (a) Metade de 844 é 422, não 421. De onde vem a diferença de uma unidade?
- (b) Deduza a expressão geral que relaciona $N(2f)$ a $N(f)$.
- (c) Para que valor de N o erro de simplesmente dividir por dois passaria a valer mais de 5 cents?

TAREFA

Exercício 2.3. Um estudante remove a tabela `nota_meios` e usa um número fixo de meios períodos — 200 — para todas as notas.

- (a) Quanto dura cada nota da escala?
- (b) Descreva o efeito musical.
- (c) Proponha uma correção que não exija a segunda tabela.

TAREFA

Exercício 2.4. Um estudante troca `uint16_t` por `uint32_t` no parâmetro de `atraso_unidades`, “para poder esperar mais”.

- (a) O que acontece com b ?
- (b) O atraso máximo obtível aumenta na proporção esperada?
- (c) Proponha uma solução melhor para atrasos longos, sem alterar o tipo.

TAREFA

Exercício 2.5. Um projeto cresce e passa a declarar 140 bytes de variáveis globais. A rotina de temporização, que estava calibrada, começa a produzir períodos cerca de 10% maiores, sem que uma única linha dela tenha sido alterada.

Explique o mecanismo e proponha duas providências.

TAREFA

Exercício 2.6. A §4.1 mostrou que `__delay_ms` é uma macro. Considere agora duas macros de corpo composto:

```
#define TOCA_A(n)  do { BUZZER ^= 1; atraso_unidades(n); } while(0)
#define TOCA_B(n)  if(1){ BUZZER ^= 1; atraso_unidades(n); }
```

- (a) Compare o código de máquina gerado pelas duas.
- (b) Exiba um trecho em que uma funciona e a outra não, e diga qual erro o compilador emite em cada caso.
- (c) Um colega argumenta que `TOCA_B` é preferível por ser mais legível. Responda.